

UNIVERSITY OF THESSALY
SCHOOL OF ENGINEERING
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

FPGA IMPLEMENTATION AND OPTIMIZATION OF THE LOCAL SEQUENCE ALIGNMENT ALGORITHM

Christos Mountakis 3801
Alexandra Tsakiri 3779

Department of Electrical & Computer Engineering
University of Thessaly

October 19, 2026

Chapter 1

Introduction

In this report, we present the design, implementation, and optimization of the Local Sequence Alignment (LSAL) algorithm on multiple computing platforms, including a traditional x86 CPU, an embedded ARM Cortex-A9 processor, and an FPGA accelerator on the ZedBoard development kit. The aim of this project was to explore the potential of reconfigurable computing, particularly using High-Level Synthesis (HLS) and OpenCL, to accelerate computationally intensive bioinformatics workloads.

Starting from a baseline software implementation on the x86 CPU, we profiled the algorithm to identify performance bottlenecks and evaluated the benefits of porting it to the embedded ARM processor available on the ZedBoard. Subsequently, we developed a hardware accelerator for the LSAL kernel using Vitis HLS, focusing on optimizations such as data-level parallelism, memory bandwidth enhancement through burst transfers, and fixed-point arithmetic to efficiently utilize FPGA resources.

The accelerator design was validated through emulation and deployed on the FPGA fabric, communicating with the embedded processor via OpenCL. Throughout this work, we analyzed the trade-offs between latency, throughput, and resource utilization, demonstrating significant performance improvements over the software implementations.

This report documents the methodology followed, the key design decisions made, and the resulting performance gains, providing insights into the challenges and opportunities of FPGA acceleration in bioinformatics applications.

Chapter 2

Software: Smith-Waterman LSAL Algorithm

2.1 Parallel LSAL Algorithm with OpenMP

Step	Description
1	Each thread initializes its local max value and position.
2	Loop through diagonals of the matrix.
3	Threads work on different elements of the current diagonal.
4	Compare characters from the two strings.
5	Compute scores from diagonal, top, and left cells.
6	Choose the highest score (or 0 if all are negative).
7	Store score and direction in matrices.
8	Update thread-local max value and position.
9	After all iterations, threads store their local best results.
10	Final step: find and store the global max position.

Listing 2.1: LSAL Algorithm without OpenMP (Sequential)

```
max_value = -INFINITY;      // Initialize max score
max_pos = -1;               // Initialize max position

for (int index = 0; index < total_cells; ++index) {
    // Compute scores from neighboring cells
    north = similarity_matrix[index - N];           // Above cell
    west = similarity_matrix[index - 1];            // Left cell
    northwest = similarity_matrix[index - N - 1];   // Diagonal cell

    // Determine if characters match or mismatch
    if (string1[col] == string2[row]) {
        match = MATCH;
    } else {
        match = MISS_MATCH;
```

```
    }

    // Calculate the cell score using max of candidate scores or zero
    val = std::max({
        north + GAP_d,
        west + GAP_i,
        northwest + match,
        0
    });

    // Update matrices
    similarity_matrix[index] = val;
    direction_matrix[index] = direction_from(val);  // Determine
       direction (assumed function)

    // Track max score and position
    if (val > max_value) {
        max_value = val;
        max_pos = index;
    }
}

// Store the position of the highest scoring cell
max_index = max_pos;
```

2.2 Comparison of Two Local Sequence Alignment Algorithms

2.2.1 Pros and Cons of Each Approach

- **Sequential Approach**
  - *Pros:*
    - Simpler and easier to implement.
    - Deterministic execution with no concurrency issues.
    - Useful as a reference model for correctness and debugging.
  - *Cons:*
    - Limited by single-core performance.
    - Not scalable for large input sequences.
    - Inefficient use of modern multi-core processors.

- **Parallel Approach with OpenMP**
  - *Pros:*
    - Significantly faster on multi-core systems.
    - Better suited for large-scale sequence alignment problems.
    - Utilizes hardware resources more efficiently.
  - *Cons:*
    - More complex implementation due to parallel traversal and synchronization.
    - Requires careful handling of shared data and reduction.
    - Slight overhead from thread management and reduction steps.

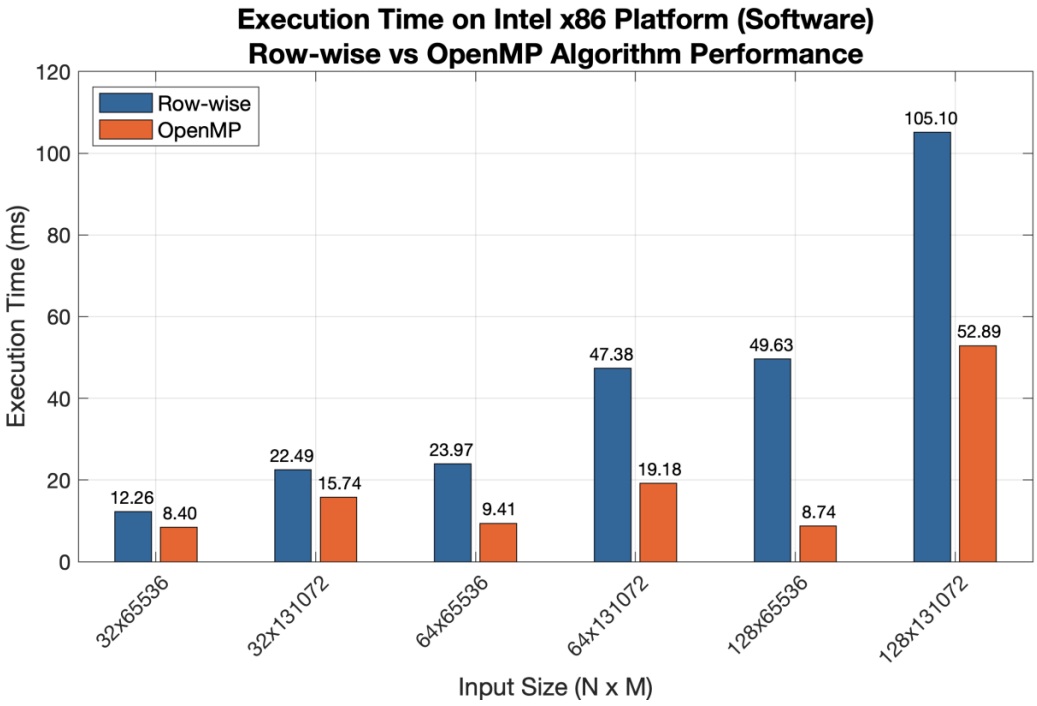


Figure 2.1: Execution time comparison for software implementations.

As shown in Figure 2.1, the OpenMP version exhibits significantly better performance, especially for larger inputs and higher values of N. Since each diagonal is divided among multiple threads, the benefits of parallelism become more pronounced as the problem size increases.

2.3 Roofline and Time Calculation

2.3.1 N=32 AND M=65536

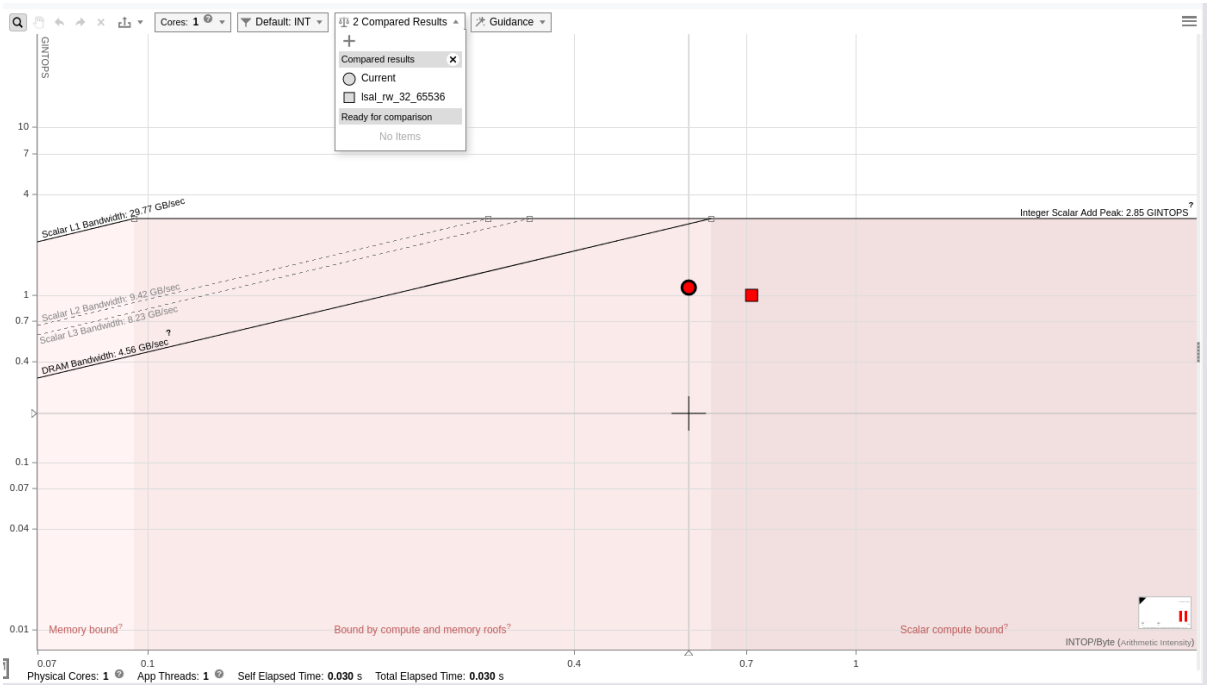


Figure 2.2: Roofline model for N=32 and M=65536.

2.3.2 N=256 AND M=137210

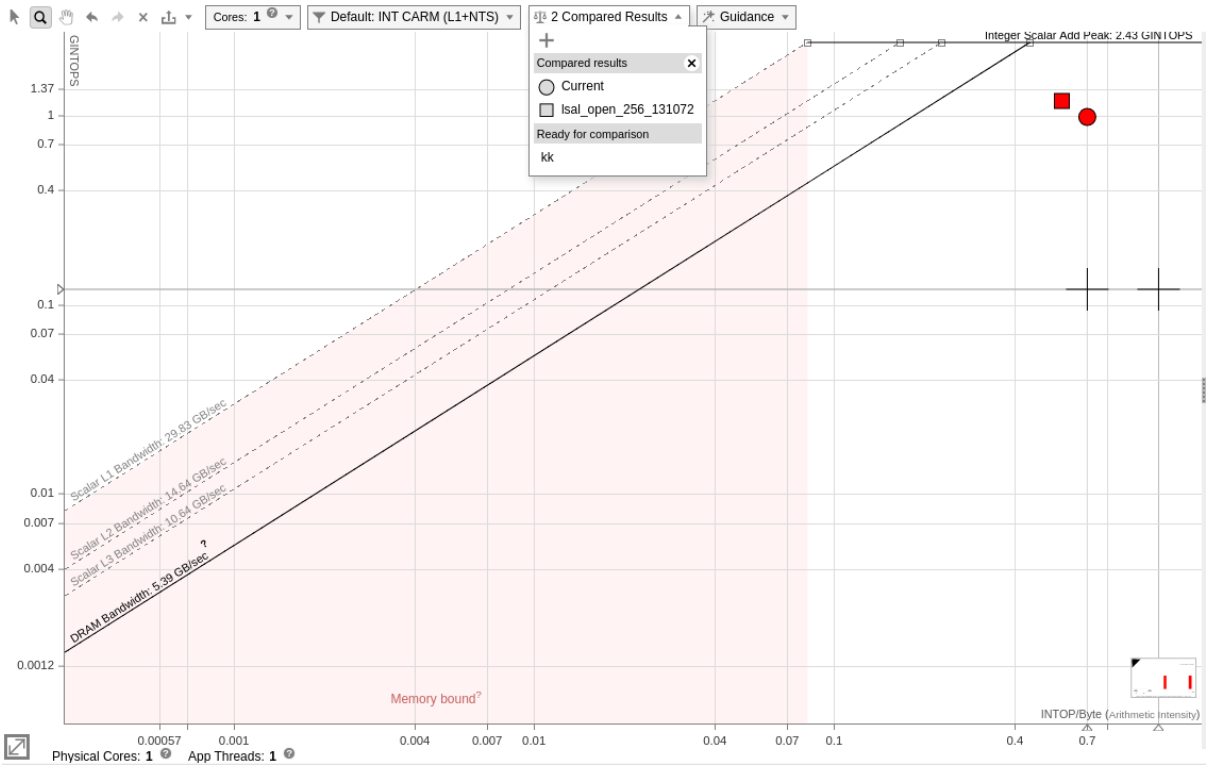


Figure 2.3: Roofline model for N=256 and M=137210.

Chapter 3

Software in Arm

The ARM software implementation reuses the same source code as the x86 version, with the exception that it is compiled without OpenMP due to the lack of multithreading support in the target environment.

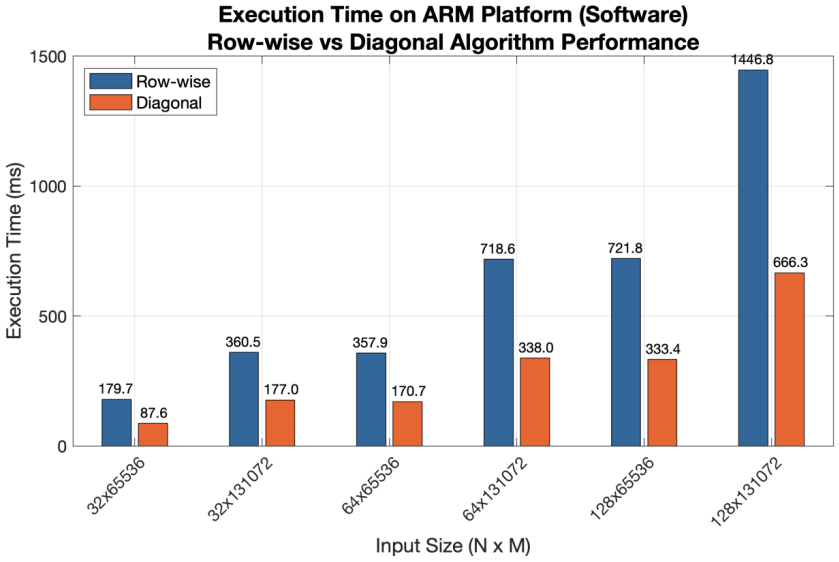


Figure 3.1: Performance comparison on the ARM architecture.

Since the diagonal algorithm is inherently single-threaded, it provides a consistent baseline across platforms. On x86, both the standard and diagonal versions show similar performance when run with a single thread. However, on ARM, the diagonal version is more than twice as fast as the standard one (Figure 3.1).

This performance difference can be attributed to architectural factors, such as:

- Better exploitation of out-of-order execution on simpler loop structures.
- More regular and predictable memory access patterns in the diagonal version.
- Reduced control-flow complexity and fewer branch mispredictions.
- Better cache utilization due to simpler data reuse, even though both versions access memory sequentially.

Chapter 4

Hardware Optimizations

4.1 Hardware Execution Without Kernel Optimizations (Baseline)

This baseline implementation runs the same kernel on the FPGA as in the ARM software version, without applying any HLS optimizations. It serves as a reference for performance comparison. The resource utilization and timing results reported by the Vitis synthesis tool provide insight into the hardware cost of this unoptimized kernel.

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
compute_matrices	II Violation	-	308750508	3.088E9	-	308750509	-	no	2	0	2850	5240	0
Diag		-	308750294	3.088E9	4709	-	65566	no	-	-	-	-	-
Compute	II Violation	-	4706	4.706E4	150	147	32	yes	-	-	-	-	-

Figure 4.1: Vitis HLS synthesis report for the unoptimized baseline implementation.

4.2 Optimization #1: Diagonal Processing and Memory Efficiency

As seen in Figure 4.1, the baseline required improvement. Optimization #1 reduces memory usage and improves access latency by combining diagonal execution with the efficient use of on-chip BRAM.

- **1D Diagonal Buffers:** Instead of allocating the full $N \times (N+M-1)$ `similarity_matrix`, we maintain only three 1D arrays of length N:
  - `diag_prevprev`: diagonal $i-2$,
  - `diag_prev`: diagonal $i-1$,
  - `diag_curr`: diagonal i.

  Since each cell depends only on its top, left, and top-left neighbors, these three diagonals are sufficient. All fit in BRAM, reducing reliance on external memory.

- **Copying Inputs to BRAM:** Input strings are copied once into BRAM before computation:

```
memcpy(string1_buf, string1, N * sizeof(char));
memcpy(string2_buf, string2, M * sizeof(char));
```

This avoids repeated DDR access during the scoring loop.

- **Buffered Direction Writes:** Instead of writing each direction immediately to `direction_matrix`, values for the current diagonal are first stored in a local buffer (`cur_dir`) and written in a single burst `memcpy`. This reduces total writes from $N \times M$ to $N + M - 1$.

- **Skipping Invalid Diagonal Regions:** On anti-diagonal i, query position j is paired with database position $i - j$, so only $j_{lo} \leq j \leq j_{hi}$ is a real cell, with $j_{lo} = \max(0, i - M + 1)$ and $j_{hi} = \min(i, N - 1)$. The first $N - 1$ diagonals are cut at the top and the last $N - 1$ at the bottom. Padding cells outside this range are skipped using:

```
int j_lo = (i >= M) ? (i - (M - 1)) : 0;
int j_hi = (i < N) ? i : N - 1;

if ((!valid_diag && !valid_up && !valid_left) || (j < j_lo) || (j >
    j_hi)) {
    val = 0;
}
```

- **Local Maximum per Diagonal:** A variable `local_max_val` tracks the maximum score within a diagonal. The global `max_value` is updated once per diagonal, supporting pipelined execution and better performance.

Summary of Benefits:

- Reduces memory footprint (no full matrix allocation).
- Eliminates repeated DDR reads by using BRAM.
- Reduces write bandwidth via diagonal buffering.
- Enables pipelined, high-throughput diagonal processing.

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM(%)	DSP(%)	FF(%)	LUT(%)	URAM(%)
compute_matrices	II Violation	-	2131125	2.131E7	-	2131126	-	no	23	0	7	35	0
Loop 1		-	32769	3.280E5	34	32	1024	yes	-	-	-	-	-
Loop 2		-	32	320.000	1	1	32	yes	-	-	-	-	-
Loop 3		-	32	320.000	1	1	32	yes	-	-	-	-	-
Diag	II Violation	-	2098144	2.098E7	32	32	65567	yes	-	-	-	-	-

Figure 4.2: Optimized diagonal-based execution flow and BRAM usage.

4.3 Optimization #2: Cyclic Array Partitioning for Parallel Access

This optimization focuses on improving parallelism and HLS tool compatibility through fine-grained BRAM access and loop restructuring, as illustrated in Figure 4.3:

```
#pragma HLS ARRAY_PARTITION variable=string1_buf cyclic factor=2
#pragma HLS ARRAY_PARTITION variable=diag_curr cyclic factor=2
#pragma HLS ARRAY_PARTITION variable=diag_prev cyclic factor=2
#pragma HLS ARRAY_PARTITION variable=diag_prevprev cyclic factor=2
#pragma HLS ARRAY_PARTITION variable=cur_dir cyclic factor=2
```

Cyclic array partitioning splits arrays into multiple independently addressable memory banks, allowing parallel access to adjacent elements in the same clock cycle. This reduces the initiation interval (II) and improves throughput.

Additionally, we restructured the loop that processes each diagonal to always run for exactly `N` iterations, even if some elements fall outside the valid region. These out-of-bound elements are safely handled using conditional logic to skip computation or set values to zero. Fixed-length loops are better optimized by Vitis HLS, enabling more predictable pipelining and scheduling.

Summary of Benefits:

- Enables parallel BRAM reads/writes, reducing loop II.
- Improves compatibility with Vitis HLS by using fixed-size loops.
- Gains performance with minimal additional resource usage.

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
compute_matrices	II Violation	-	1278754	1.279E7	-	1278755	-	no	62	0	8022	18843	0
Loop 1		-	32769	3.280E5	34	32	1024	yes	-	-	-	-	-
Loop 2		-	32	320.000	1	1	32	yes	-	-	-	-	-
Loop 3		-	32	320.000	1	1	32	yes	-	-	-	-	-
Diag	II Violation	-	1245773	1.246E7	20	19	65567	yes	-	-	-	-	-

Figure 4.3: Impact of cyclic array partitioning.

4.4 Optimization #3: Sliding Buffer, Partitioning, and Unrolling

During the hardware implementation of the LSAL kernel, a significant memory bandwidth bottleneck was identified related to the second input string array, `string2`. Specifically, the available memory bandwidth was insufficient to efficiently transfer and process the entire array at each diagonal computation step.

The original implementation attempted to access large portions of `string2` at each diagonal iteration, resulting in redundant memory reads and suboptimal memory utilization. This created latency issues and frequent stalls in the HLS pipeline due to memory bandwidth constraints. Achieving an initiation interval (II) of 1 with this approach would have required a high partitioning factor on `string2`, significantly increasing BRAM usage.

To address this, our design incorporates a **sliding buffer** of size `N` (32 in this case), which temporarily stores the relevant diagonal segment of `string2`. This buffer is dynamically updated at each iteration, significantly reducing memory access requirements (Figure 4.4). By avoiding full partitioning of the input array, we minimize BRAM usage

while enabling the pipeline to achieve an initiation interval (II) of 1 with positive timing slack, and consistent performance even if forced to II = 2.

Each new diagonal shifts the buffer left by one position and inserts `string2[i]` at the right end. After the last character of `string2` (diagonals $i \geq M$), the buffer keeps shifting and inserts a padding 0 instead. This keeps the invariant `sliding_buf[N - 1 - j] = string2[i - j]` on every diagonal, including the last $N-1$, where the valid cells are at the bottom of the diagonal. The padding is never compared, since those positions are outside $[j_{lo}, j_{hi}]$. This approach reduces memory reads from `N` to just **1 read per diagonal**.

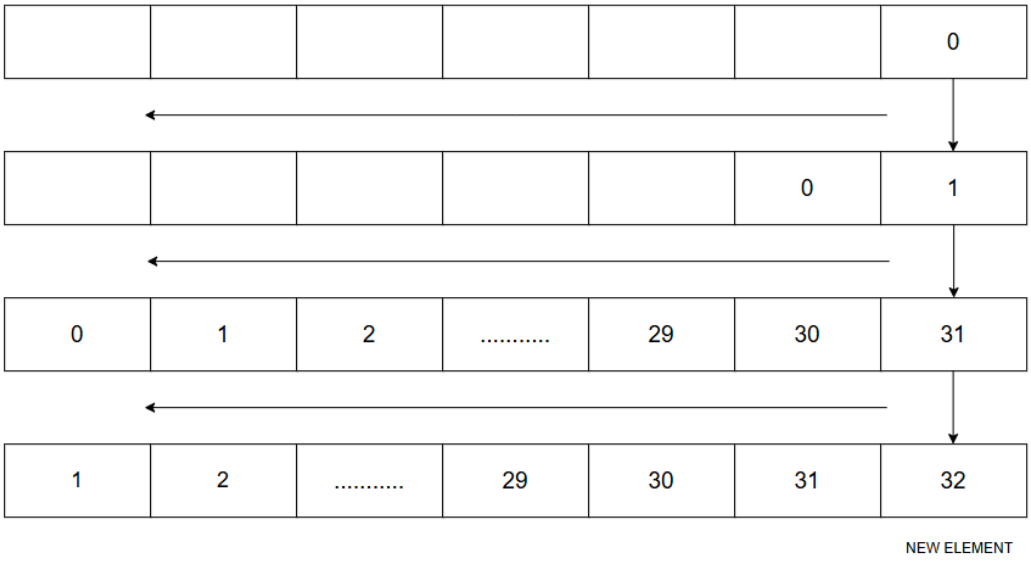


Figure 4.4: Dynamic updating of the sliding buffer.

```
char_t sliding_buf[N] = {0};
#pragma HLS ARRAY_PARTITION variable=sliding_buf cyclic factor=2

if (i != 0) {
    for (int k = 0; k < N - 1; ++k)
        #pragma HLS UNROLL
        sliding_buf[k] = sliding_buf[k + 1];
    sliding_buf[N - 1] = (i < M) ? string2_char(i) : char_t(0);
}
```

To support diagonal processing, the sliding buffer is read in reverse, from the last to the first character:

```
char_t b = sliding_buf[(N - 1) - j];
```

Since `sliding_buf` holds 32 characters, reverse indexing aligns it properly with the diagonal traversal for correct character comparisons.

4.4.1 Advantages

- Reduces memory reads from 32 to 1 per diagonal step.
- Minimizes memory bandwidth usage.

- Enables more aggressive loop unrolling and parallelization.
- Enables deeper pipelining and more efficient resource usage.

4.5 Loop Unrolling Optimization

When an outer loop is pipelined, Vitis HLS automatically attempts to fully unroll the inner loops. While adding `#pragma HLS UNROLL` might seem redundant in theory, explicitly defining it guides the compiler's scheduling heuristics, resulting in better resource allocation and routing on the ZedBoard. Because of full unrolling, all `N` elements are processed simultaneously in a single cycle, rather than just a few elements per cycle.

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
compute_matrices		-	139590	1.396E6	-	139591	-	no	62	0	11137	19319	0
Loop 1		-	8193	8.193E4	10	8	1024	yes	-	-	-	-	-
Loop 2		-	32	320.000	1	1	32	yes	-	-	-	-	-
Loop 3		-	32	320.000	1	1	32	yes	-	-	-	-	-
Loop 4		-	31	310.000	1	1	31	yes	-	-	-	-	-
Diag		-	131151	1.312E6	20	2	65567	yes	-	-	-	-	-

Figure 4.5: Synthesis results following sliding buffer and loop unrolling implementations.

4.6 Optimization #4: Arbitrary Precision Optimization

Using arbitrary precision types (`ap_uint`) is a critical optimization for efficient resource utilization on the FPGA. Specifically:

- **BRAM usage is reduced** because we avoid storing unnecessary bits; only the required bits are used.
- **Previous timing violations (with II=1) are resolved**, as narrower bitwidths simplify routing and reduce critical path delays.
- While this does not directly decrease the execution time in terms of clock cycles, the reduced hardware footprint eases routing congestion, thereby allowing the design to meet timing constraints at higher clock frequencies.
- **Score widths are sized to the query length.** The best possible score is a full-length match, $2N$, so the signed intermediate scores (`score_int_t`) need $\lceil \log_2(2N) \rceil + 2$ bits: `ap_int<8>` for $N = 32$. One bit fewer wraps a perfect match to a negative value.

4.6.1 Burst Memory Transfers with 32-Character Elements

Each element of `string1_buf` and `string2_buf` stores 32 characters packed into 64 bits (assuming `char_t = ap_uint<2>`). This layout enables efficient burst memory transfers, as multiple characters are transferred together in a single burst, reducing transfer overhead and improving throughput:

```
memcpy(string1_buf, string1, N * sizeof(ap_uint<64>) / 32);
memcpy(string2_buf, string2, M * sizeof(ap_uint<64>) / 32);
```

Because of this alignment and burst transfer, the partition factor required in hardware is reduced, leading to lower BRAM usage and better overall performance.

4.6.2 Trade-off Between Burst Width and Bit Extraction Efficiency

The 64-bit word size represents a balance between maximizing burst transfer efficiency and maintaining manageable bit extraction complexity. Each element holds 32 characters, each occupying 2 bits, resulting in 64 bits per word.

Character extraction from a packed 64-bit word uses the `.range()` method as follows:

```
ap_uint<6> offset_bits = (j % 32) * 2;
char_t a = string1_buf[j / 32].range(offset_bits + 1, offset_bits);
```

Here, 6 bits suffice for the offset calculation since $2^6 = 64$ bits cover the entire word.

While wider word sizes (such as 128, 256, or 512 bits) could increase burst transfer bandwidth by moving more data per cycle, they also increase the complexity and latency of bit extraction. Extracting a 2-bit character from a larger word requires a wider `.range()` operation, which can slow down access and add hardware overhead.

Thus, there is an inherent trade-off between:

- Using wider words to optimize burst memory throughput by transferring more data simultaneously.
- Keeping word widths smaller to enable faster, simpler, and lower-latency bit-range extraction for individual characters.

Our choice of 64-bit words achieves an efficient compromise, enabling burst transfers with good throughput while maintaining fast and simple character access logic.

4.6.3 Direction Matrix: Bitwidth Minimization

In the direction matrix, each word encodes one diagonal. The write operation is performed as follows:

```
dir_mat_t dir_word = 0;
PackLoop:
for (int k = 0; k < N; ++k) {
    #pragma HLS UNROLL
    dir_word.range(k * 2 + 1, k * 2) = dir_curr[k];
}
```

Since only write access is required here (no complex reads), the costly bit-level lookups needed elsewhere are avoided. Writing alone determines the minimal bitwidth used, which is 2 bits per position. This significantly reduces resource usage.

4.6.4 Loop Pipelining

The outer loop is pipelined with:

```
#pragma HLS PIPELINE II=1
```

This achieves an initiation interval (II) of 1, allowing the pipeline to start processing a new diagonal every clock cycle. Vitis HLS already pipelines the loop with II=1, but adding this pragma helps the compiler better optimize the design for FPGA synthesis (Figure 4.6).

Modules & Loops	Issue Type	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
compute_matrices	-	-	67777	6.780E5	-	67778	-	no	12	0	4806	9726	0
Loop 1	-	-	2049	2.049E4	3	1	2048	yes	-	-	-	-	-
Diag	-	-	65583	6.560E5	18	1	65567	yes	-	-	-	-	-

Figure 4.6: Final Vitis HLS report incorporating arbitrary precision optimizations.

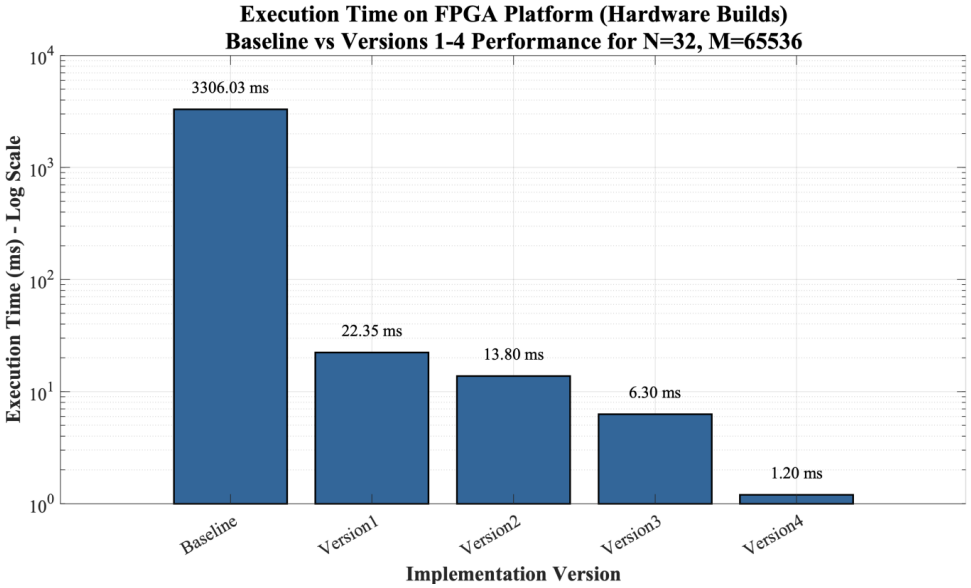


Figure 4.7: Cumulative performance of all hardware optimization stages.

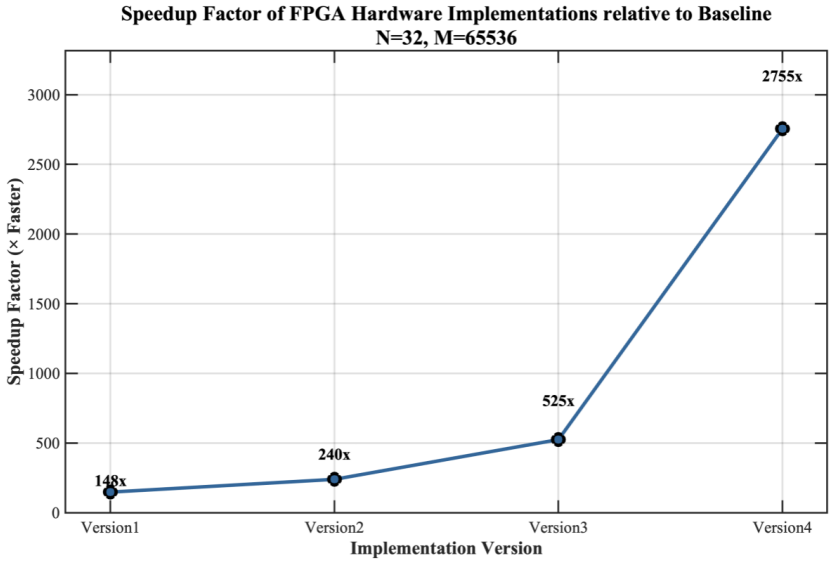


Figure 4.8: Speedup of each hardware version over the unoptimized hardware baseline.

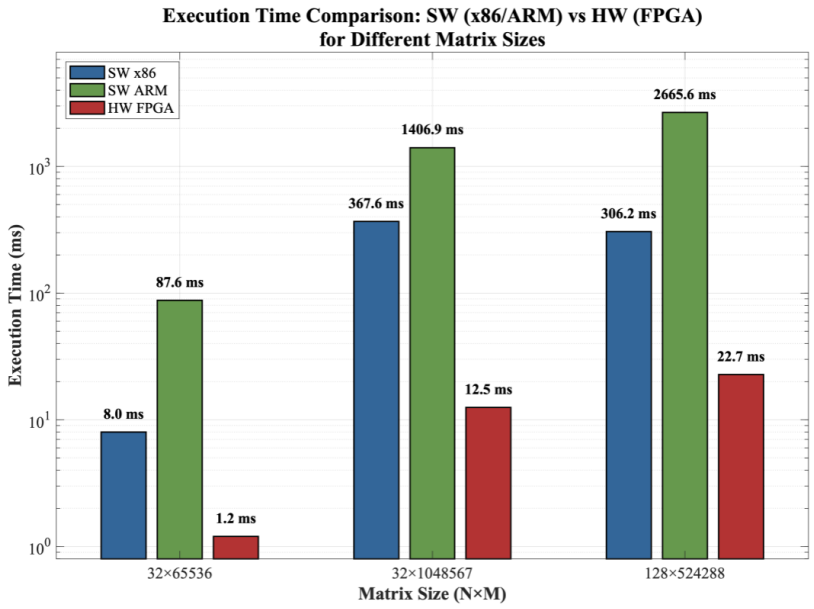


Figure 4.9: Comparison of execution times across all target environments.

Latency

		solution1	solution2	solution3	solution4	solution5
Latency (cycles)	min	9966246	2294984	1278754	139590	67777
	max	308750508	2294984	1278754	139590	67777
Latency (absolute)	min	99.662 ms	22.950 ms	12.788 ms	1.396 ms	0.678 ms
	max	3.088 sec	22.950 ms	12.788 ms	1.396 ms	0.678 ms
Interval (cycles)	min	9966247	2294985	1278755	139591	67778
	max	308750509	2294985	1278755	139591	67778

Utilization Estimates

	solution1	solution2	solution3	solution4	solution5
BRAM_18K	2	36	62	62	12
DSP	0	0	0	0	0
FF	2850	4759	8022	11137	4806
LUT	5240	13390	18843	19319	9726
URAM	0	0	0	0	0

Figure 4.10: Latency and resource utilization of the HLS solutions.

Summary

Through a series of hardware-oriented optimizations, including diagonal reformulation, BRAM caching, loop pipelining, cyclic array partitioning, and arbitrary precision arithmetic, we transformed the original dynamic programming algorithm into a highly parallel and memory-efficient implementation tailored for FPGA acceleration. These optimizations effectively exploited on-chip memory resources and enabled fine-grained parallelism, significantly improving throughput and reducing latency.

As shown in Figures 4.7 and 4.8, optimizations reduced the execution time of the hardware kernel from 3306 ms in the unoptimized baseline to 1.2 ms, a **speedup of 2755**×. Compared with the software implementations for $N = 32$ and $M = 65536$ (Figure 4.9), the final FPGA design is **73**× **faster than the ARM** (87.6 ms) and 6.7× faster than the x86 CPU (8.0 ms). This demonstrates the substantial performance potential of HLS-based hardware acceleration when the algorithm's structure and memory access patterns are carefully adapted to leverage the capabilities of the target hardware platform.